

Comparative Analysis of Supervised Learning Models on Hotel Booking Demand and US Accidents Datasets

Abstract

This paper presents a comparative study of four widely used machine learning models: decision trees, k -nearest neighbors, neural networks, and support vector machines applied to two real-world datasets: Hotel Booking Demand (classification) and US Accidents (regression). Each model was implemented within reproducible pipelines, with consistent preprocessing, stratified splits, and evaluation using task-appropriate metrics. Beyond accuracy, we assessed runtime and memory usage to capture practical deployment considerations, and we employed diagnostic plots to analyze learning behavior, calibration, and sensitivity to hyperparameters.

Across both tasks, neural networks delivered the strongest overall performance, achieving the highest ROC-AUC in classification and the lowest RMSE in regression, particularly when regularization and early stopping were applied. Decision trees and k NN provided competitive baselines with strong interpretability, though their accuracy plateaued relative to neural networks. SVMs showed mixed results: the RBF kernel performed reasonably well for classification but scaled poorly in regression, while the linear variant failed to converge on the accident dataset. These findings highlight the trade-offs between interpretability, scalability, and predictive power, underscoring the importance of aligning model choice with both task requirements and resource constraints.

1 Introduction and Related Work

Machine learning methods are increasingly applied to large, heterogeneous datasets in order to extract predictive insights and support decision making. In this report we evaluate and compare a range of supervised learning algorithms across two distinct domains: hotel booking demand and traffic accident duration. The goal is to assess not only predictive accuracy but also computational efficiency, interpretability, and robustness under realistic constraints.

For the **Hotel Booking Demand** dataset, the prediction task is binary classification: determining whether a reservation will be cancelled. This dataset, first introduced by Antonio et al. [1], has become a widely used benchmark for studying imbalanced classification, feature encoding strategies, and the trade-off between interpretable and complex models. Cancellation prediction is of practical importance to hotel operators, as it enables dynamic pricing, overbooking strategies, and improved resource allocation.

For the **US Accidents** dataset, the task is regression: predicting the duration of an accident in minutes based on contextual and environmental features. The dataset, described by Moosavi et al. [2], is one of the largest publicly available traffic accident data, containing millions of records with temporal, spatial, and weather attributes. Accurate duration prediction is critical for traffic management systems, emergency response, and traveler information services.

Across both datasets, we implement and compare four representative families of models: decision trees, support vector ma-

chines (linear and kernelized), k -nearest neighbors, and neural networks (with both shallow and deep architecture). These algorithms span a spectrum of complexity and inductive bias: from interpretable tree structures to nonparametric instance-based learning and high-capacity neural architectures. By applying consistent preprocessing pipelines and evaluation protocols, we aim to highlight the strengths, limitations, and computational trade-offs of each approach.

Hypothesis: Based on prior work in supervised learning, we hypothesize that neural networks will achieve the strongest predictive performance on both tasks, due to their ability to capture nonlinear feature interactions and scale to large datasets. However, we expect decision trees and k -nearest neighbors to provide competitive baselines with greater interpretability, while support vector machines will show mixed results: kernelized variants may perform well on the hotel classification task but will face scalability challenges on the accident regression task. We further hypothesize that these differences will be reflected not only in accuracy metrics but also in runtime, memory usage, and generalization behavior observed in learning and validation curves.

Our contributions are threefold: First, we provide a systematic comparison of classical and modern machine learning methods on two large, real-world datasets with different prediction targets (classification vs. regression). Second, we document the impact of preprocessing choices, hyperparameter tuning, and compute budgets on model performance and scalability. Third, we analyze learning curves, validation curves, and training dynamics to illustrate algorithmic behavior, including cases where models fail to converge or generalize.

The remainder of this report is organized as follows. Section 2 describes the datasets and preprocessing steps. Section 3 outlines the models and training protocols. Section 4 presents quantitative results and diagnostic plots. Section 5 discusses trade-offs and practical implications, and Section 6 concludes with recommendations and future directions.

2 Data and Preprocessing

2.1 Hotel Booking Demand Dataset

The Hotel Booking Demand dataset introduced by Antonio et al. [1] contains over 119k booking records from a Portuguese hotel group. The prediction task is binary classification: determining whether a reservation will be cancelled. From a machine learning perspective, this dataset is compelling due to its moderate class imbalance, high-cardinality categorical features, and the presence of temporal and behavioral signals. It offers a realistic challenge in encoding, calibration, and generalization, especially for models sensitive to feature distributions.

We removed leakage-prone columns such as `reservation_status` and `reservation_status_date`, which directly encode the outcome. High-cardinality categorical features including `country`, `agent`, and `company` were frequency encoded to avoid excessive dimensionality and overfitting. All remaining

categorical variables were encoded consistently, and missing values were imputed with the placeholder “Unknown.”

The original dataset contained 32 columns in total, including the target variable. After removing the target and a small number of leakage-prone columns, the dataset contained 30 columns. Of these, 29 served as predictors in the final feature matrix, with `is_canceled` defined separately as the classification target. Features were standardized prior to training, and stratified sampling was used to preserve class balance across training, validation, and test splits. Most models were trained on the full dataset, except for Kernel SVM, which were capped at 20k rows due to runtime constraints. For k -nearest neighbors, distance weighting was used to mitigate the impact of class imbalance and improve sensitivity to rare cancellation patterns.

2.2 US Accidents Dataset

The US Accidents dataset [2] is one of the largest publicly available traffic accident data, with close to 8 millions of records collected from multiple states. The prediction task is regression: estimating the duration of an accident in minutes based on contextual and environmental features. This dataset presents several challenges for machine learning practitioners, including noisy labels, skewed duration distributions, high dimensionality, and potential feature leakage. It is particularly well-suited for studying scalability, regularization, and runtime trade-offs across model families.

We defined the target variable `Duration` as the difference between `Start_Time` and `End_Time`, and excluded records with durations longer than 360 minutes to remove extreme outliers. To prevent data leakage and reduce redundancy, the following columns were dropped: `ID`, `Source`, `End_Time`, `Start_Time`, `Street`, `Zipcode`, `Airport_Code`, `City`, `Country`, `State`, and `Timezone`. Temporal features (`Hour`, `DayOfWeek`, `Month`) were extracted from the start time. Finally, all numerical features, including the target, were cast to `float32` to ensure consistency and reduce memory usage across models.

Categorical features were frequency encoded, missing values were imputed with column means, and all features were standardized. The initial feature matrix contained 46 columns. After preprocessing, the dataset contained 39 columns in total. Of these, 38 served as predictors in the final feature matrix, with `Duration` defined separately as the regression target. Sampling caps were applied to respect algorithmic constraints: decision trees were trained on $\sim 1\text{M}$ rows, kernel SVMs on 100k rows (with a 25k coarse search), and k -nearest neighbors on 250k rows with evaluation on 25k. Neural networks were trained on a 5M row subsample, consistent with the compute budget. Data splits followed an 70/15/15 train/validation/test protocol, with stratification where applicable.

3 Methods

3.1 Hotel Booking Demand

3.1.1 Decision Trees

For the hotel booking classification task, we implemented a decision tree classifier using `scikit-learn`’s `DecisionTreeClassifier`, integrated into a preprocessing pipeline that included target encoding and feature

scaling. High-cardinality categorical features such as `country`, `agent`, and `company` were encoded using a smoothed target encoder, while lower-cardinality features were encoded without smoothing. All numeric features were standardized using `StandardScaler` with `with_mean=False` to preserve sparse representations.

Because the dataset is moderately imbalanced, we optimized primarily for the F1 score, which balances precision and recall across classes. ROC-AUC and PR-AUC were also reported to capture calibration and discriminative ability under imbalance, while accuracy was treated as a secondary metric.

Hyperparameter tuning was conducted via grid search across five dimensions: tree depth, minimum samples per leaf, minimum samples per split, feature subset size, and cost-complexity pruning α . The search space included depths up to 18, leaf sizes of 50 to 200, and pruning alphas ranging from 0 to 10^{-3} . A total of 432 hyperparameter combinations were evaluated using 3-fold stratified cross-validation, resulting in 1296 model fits. Validation curves indicated that performance plateaued beyond depth 14, with overfitting at higher depths. We therefore capped the maximum depth at 18 and selected the final α value by re-running along the pruning path and choosing the configuration that minimized validation error. The best configuration used depth 18, minimum leaf size of 50, minimum split size of 100, feature subset size of 50%, and no pruning.

The final model achieved a cross-validated F1 score of 0.804 and produced a tree with 990 leaf nodes. On the held-out test set of 23,878 examples, it reached an overall accuracy of 86%, with class-wise precision of 0.92 for non-cancellations and 0.78 for cancellations. Recall was 0.86 and 0.87 respectively, yielding F1 scores of 0.89 and 0.82. The model also achieved a ROC-AUC of 0.947 and a PR-AUC of 0.920, indicating strong discriminative performance even under class imbalance.

Training completed in 5.08 minutes with a peak RAM usage of 0.323 GB, measured for the main Python process. Feature importance analysis revealed that `deposit_type`, `lead_time`, and `previous_cancellations` were among the most influential predictors of cancellation.

Note: All experiments were run on the same system (16 threads, 61.8 GB RAM, Windows 10) with parallelism enabled via `n_jobs > 1`. These specifications apply throughout the study.

3.1.2 Support Vector Machines

Support vector machines were implemented using both linear and kernelized variants to assess the impact of decision boundary flexibility on classification performance. The preprocessing pipeline included three parallel transformations: high-cardinality categorical features (`country`, `agent`, `company`) were encoded using a smoothed target encoder; low-cardinality categorical features were one-hot encoded with unknown handling; and numeric features were standardized using `StandardScaler`. These transformations were combined using a `ColumnTransformer` and applied consistently across both SVM variants.

Because the hotel dataset is imbalanced, we prioritized the F1 score as the primary optimization metric, since it balances precision and recall across classes. ROC-AUC and PR-AUC were also reported to capture calibration and discriminative ability, while accuracy was treated as a secondary measure.

For the linear SVM, we used `LinearSVC` with balanced class

weights and disabled the dual formulation to improve efficiency on dense data. Hyperparameter tuning was performed via grid search over regularization strength ($C \in \{0.1, 1, 10\}$) and maximum iteration count ($\text{max_iter} \in \{5000, 10000, 20000\}$). Validation curves showed that performance plateaued around $C = 1$, with higher values offering no consistent gains. We therefore selected $C = 1$ with 5000 iterations, achieving a cross-validated F1 score of 0.766. On the held-out test set, the linear SVM reached an overall accuracy of 82%, with class-wise F1 scores of 0.85 for non-cancellations and 0.76 for cancellations. Training completed in 0.51 minutes with a peak RAM usage of 0.403 GB.

For the kernel SVM, we used `SVC` with an RBF kernel, balanced class weights, and probability calibration enabled. The grid search explored regularization strength ($C \in \{0.5, 2, 8\}$) and kernel bandwidth ($\gamma \in \{\text{scale}, 1/d, 2/d\}$, where d is the number of features). Iterative tuning showed that validation F1 improved steadily up to $C = 8$ but plateaued beyond that, while $\gamma = \text{scale}$ provided the most stable calibration. The final configuration ($C = 8$, $\gamma = \text{scale}$) yielded a cross-validated F1 score of 0.796. On the test set, the RBF SVM achieved an accuracy of 85%, with F1 scores of 0.88 and 0.80 for the two classes. It also produced a ROC-AUC of 0.924 and a PR-AUC of 0.893, indicating strong calibration and discriminative power. Training required 8.64 minutes and consumed 0.480 GB of RAM.

Overall, the kernel SVM outperformed its linear counterpart in both accuracy and calibration metrics, though at significantly higher computational cost. The results highlight the trade-off between scalability and decision boundary flexibility in SVM-based classifiers.

3.1.3 k -Nearest Neighbors

The k -nearest neighbors (kNN) classifier was implemented using `scikit-learn`'s `KNeighborsClassifier` with brute-force search and parallel processing enabled. The preprocessing pipeline applied three transformations: high-cardinality categorical features were encoded using a smoothed target encoder; low-cardinality categorical features were one-hot encoded with unknown handling; and numeric features were standardized using `StandardScaler`. These transformations were combined using a `ColumnTransformer` and applied consistently across all folds and configurations.

Because the dataset is moderately imbalanced, we optimized primarily for the F1 score to balance precision and recall across classes. ROC-AUC and PR-AUC were also reported to capture calibration and discriminative ability, while accuracy was treated as a secondary measure.

Hyperparameter tuning was performed using grid search over three dimensions: number of neighbors ($k \in \{3, 5, 11, 21\}$), weighting scheme (uniform vs. distance), and distance metric (euclidean vs. manhattan). Validation curves showed that performance improved steadily as k increased, with variance reduced at larger neighborhood sizes. Beyond $k = 21$, gains plateaued, and distance weighting consistently outperformed uniform weighting by improving sensitivity to minority-class cancellations. The Manhattan metric provided slightly better validation F1 than Euclidean, likely due to its robustness to high-cardinality encoded features. Based on these observations, we selected $k = 21$, distance weighting, and Manhattan distance, yielding a cross-validated F1 score of 0.797. Training completed in

3.25 minutes with a peak RAM usage of 0.598 GB.

On the held-out test set of 23,878 examples, the final kNN model achieved an overall accuracy of 86%. Class-wise precision was 0.87 for non-cancellations and 0.84 for cancellations, with corresponding recall values of 0.91 and 0.77. The resulting F1 scores were 0.89 and 0.80, respectively. The model also achieved a ROC-AUC of 0.928 and a PR-AUC of 0.907, indicating strong discriminative performance and calibration. Prediction on 25,000 examples required 20.1 seconds, highlighting the computational cost of instance-based inference at scale.

The results demonstrate that kNN can deliver competitive accuracy and calibration on this task, particularly when tuned with distance weighting and Manhattan metrics. However, its prediction latency and memory footprint make it less suitable for real-time deployment scenarios.

3.1.4 Neural Networks

We implemented feedforward neural networks in `TensorFlow/Keras` to evaluate high-capacity nonlinear models. Two architectures were tested: a shallow-wide network with two hidden layers of 512 units, and a deep-narrow network with four layers (256, 256, 128, 128). Hidden layers used ReLU with He initialization, L2 regularization ($\lambda = 10^{-4}$), and dropout (0.2). The output layer applied a sigmoid activation for binary classification.

Preprocessing included target encoding for high-cardinality categorical features (country, agent, company), one-hot encoding for low-cardinality features, and standardization of numeric columns via a `ColumnTransformer` and final `StandardScaler`. Because the dataset is imbalanced, we optimized primarily for F1, with ROC-AUC and PR-AUC reported for calibration and accuracy treated as secondary.

Models were trained with SGD + momentum (0.01, 0.9), binary cross-entropy loss, and early stopping (patience = 3). Iterative refinement showed that very small batches caused unstable convergence, while very large ones slowed training without gains; validation curves plateaued after 6–8 epochs. The shallow-wide model trained in 0.11 minutes (1.06 GB RAM), and the deep-narrow in 0.08 minutes (1.10 GB).

On the 23,878-example test set, both architectures reached 86% accuracy. The shallow model achieved F1 scores of 0.89 (non-cancellations) and 0.80 (cancellations), with ROC-AUC 0.934 and PR-AUC 0.905. The deep model offered marginal gains (ROC-AUC 0.938, PR-AUC 0.910).

An activation function study (shallow-wide architecture) compared ReLU, GELU, Swish, and Tanh. Tanh achieved the best weighted F1 (0.812), followed by ReLU (0.804), with GELU and Swish slightly worse. This suggests activation choice can influence calibration and generalization, even in relatively shallow networks.

3.2 US Accidents

3.2.1 Decision Trees

For the accident duration regression task, we implemented a decision tree regressor using `scikit-learn`'s `DecisionTreeRegressor`. The preprocessing pipeline included one-hot encoding for categorical features and median imputation for numeric columns, applied via a `ColumnTransformer`. Sparse output was preferred to

reduce memory usage, and transformations were fit exclusively on the training set to prevent leakage.

The target variable `Duration` was defined as the difference between `Start_Time` and `End_Time`, measured in minutes. Records with missing or extreme durations (greater than 360 minutes) were excluded. Temporal features such as hour, day of week, and month were extracted from the start time, and post-outcome columns including `End_Time`, `Street`, and `Description` were removed. The final dataset was sampled to 1,005,000 rows to balance coverage and computational feasibility.

Because accident durations are continuous and exhibit skewed distributions with outliers, we evaluated models using RMSE as the primary metric, since it penalizes large errors more heavily. MAE was also reported to provide a more interpretable measure of average error in minutes, while R^2 was included to capture explained variance.

Initial model configuration included a maximum depth of 18, minimum leaf size of 100, minimum split size of 200, and feature subsampling via `max_features = sqrt`. To regularize the tree and prevent overfitting, we performed cost-complexity pruning using the built-in pruning path. Eight candidate values of the pruning parameter `ccp_alpha` were selected via stratified sampling of the pruning path, and separate models were trained for each. Validation curves showed that RMSE decreased steadily with pruning up to $\alpha \approx 0.02$, after which performance plateaued. Based on this iterative tuning, we selected `ccp_alpha = 0.0238` as the final configuration.

The final pruned tree had a depth of 18, 1236 leaf nodes, and 2471 total nodes. On the validation set, it achieved a mean absolute error (MAE) of 41.39 minutes, RMSE of 64.35, and R^2 of 0.400. On the test set, performance was consistent with an MAE of 41.40, RMSE of 64.47, and R^2 of 0.397. Training completed in 5.56 minutes with a peak RAM usage of 3.141 GB, measured for the main process. Feature importance analysis revealed that `Distance(mi)` and `Wind_Chill(F)` were the most influential predictors, followed by wind direction, temperature, and severity.

3.2.2 Support Vector Machines

Support vector machines were evaluated using both linear and kernelized variants to assess their suitability for large-scale regression. The preprocessing pipeline included frequency encoding for categorical features, median imputation for numeric columns, and standardization of all predictors. The target variable `Duration` was cast to `float32`, and stratified sampling was performed on binned target values to ensure balanced splits across duration ranges.

Because accident durations are continuous and skewed with outliers, we prioritized RMSE as the primary evaluation metric, since it penalizes large errors more heavily. MAE was also tracked to provide an interpretable measure of average error in minutes, while R^2 was reported to capture explained variance relative to the baseline.

For the linear variant, we used `SGDRegressor` wrapped in a `TransformedTargetRegressor` to scale both inputs and outputs. The model was trained on 1 million examples using early stopping and a fixed regularization strength of $\alpha = 10^{-4}$. Despite multiple tuning attempts, validation curves showed divergence with RMSE values exploding and R^2 dropping far below

zero, confirming that the linear hypothesis space could not capture the nonlinear and heteroscedastic structure of accident durations. On the validation set, it produced an RMSE exceeding 2.2×10^{12} and an R^2 of approximately -7×10^{20} . Training completed in 1.75 seconds with a peak RAM usage of 1.17 GB. These results suggest that linear SVMs are ill-suited for this task.

To contextualize model performance, we also trained a dummy regressor that predicts the mean duration. This baseline achieved a validation RMSE of 83.28, establishing a reference point that any viable model should surpass.

For the kernel SVM, we used `SVR` with an RBF kernel and a two-stage tuning strategy. In Stage 1, a coarse grid search was performed on a 25,000-row subsample using 2-fold cross-validation. Validation curves indicated that performance improved steadily as C increased up to 8, after which gains plateaued, while $\gamma = \text{scale}$ provided the most stable calibration compared to $\gamma = 1/d$ or $2/d$. In Stage 2, we validated this configuration and the heuristic fallback ($\gamma = 1/d$) on a 100,000-row hold-out split. The final model ($C = 8$, $\gamma = \text{scale}$) achieved an RMSE of 76.97 and R^2 of 0.151, outperforming the dummy baseline but falling short of tree-based methods. Training required 9.96 minutes and consumed 0.747 GB of RAM.

Overall, kernel SVMs demonstrated moderate predictive power and acceptable calibration, but their scalability remains a concern. The linear variant failed to generalize, reinforcing the need for nonlinear modeling in this domain.

3.2.3 k -Nearest Neighbors

We implemented a k -nearest neighbors (kNN) regressor using `scikit-learn`'s `KNeighborsRegressor` with brute-force search and parallel processing. Preprocessing included frequency encoding for categorical features, median imputation for numeric columns, and standardization via `StandardScaler`. The dataset was capped at 250,000 rows for tractable training, with stratified splits preserving balance across duration bins.

Because accident durations are continuous and skewed with outliers, RMSE was the primary evaluation metric, with MAE reported for interpretability and R^2 for explained variance. Hyperparameter tuning explored neighbors ($k \in \{3, 5, 11, 21\}$), weighting (uniform vs. distance), and distance metric (euclidean vs. manhattan). Validation curves showed performance improved with larger k but plateaued beyond 21; distance weighting consistently outperformed uniform, and Manhattan distance yielded slightly lower RMSE than Euclidean. The final configuration ($k = 21$, distance weighting, Manhattan) achieved RMSE 65.30 and R^2 0.391. Prediction on 25,000 rows required 22.5 seconds, while total training and tuning exceeded 15 minutes. Peak RAM usage reached 4.25 GB, reflecting the memory-intensive nature of brute-force neighbor searches.

Overall, kNN delivered competitive accuracy, particularly with distance weighting and Manhattan metrics, but its latency and memory footprint limit suitability for real-time deployment. Unlike parametric models, kNN must store and scan the full training set at inference, making it computationally costly at scale.

3.2.4 Neural Networks

We implemented feedforward neural networks in `TensorFlow/Keras` to evaluate high-capacity nonlinear models for accident duration regression. Two architectures were tested: a shal-

low-wide network with two hidden layers of 512 units, and a deep-narrow network with four layers (256, 256, 128, 128). Hidden layers used ReLU with He initialization, L2 regularization ($\lambda = 10^{-4}$), and dropout (0.2). The output layer applied a linear activation for continuous predictions.

Preprocessing included frequency encoding for categorical features, mean imputation for numeric columns, and standardization via `StandardScaler`. The dataset was sampled to 5 million rows to capture rare conditions and long-duration events, with stratified splits for training, validation, and test sets. All transformations were fit exclusively on the training data to prevent leakage.

Because accident durations are continuous and skewed with outliers, RMSE was the primary evaluation metric, with MAE reported for interpretability and R^2 for explained variance. Models were trained with SGD + momentum (0.01, 0.9), mean squared error loss, and early stopping (patience = 3). Batch size was set to 1024 with a 15-epoch cap. Iterative refinement showed that very small batches caused unstable convergence, while very large ones slowed training without gains; validation curves plateaued after 7–9 epochs. The shallow-wide model trained in 8 minutes (1.05 GB RAM), and the deep-narrow in 11 minutes (1.10 GB).

On the validation set, the shallow model achieved RMSE 55.98 and R^2 0.548, while the deep model improved slightly to RMSE 55.13 and R^2 0.562. These results represent the best performance among all models evaluated, showing that neural networks capture nonlinear relationships and heteroscedasticity in accident durations. The deep architecture offered marginal gains, suggesting that additional depth can improve generalization when regularization and dropout are properly configured.

4 Results

This section presents the comparative performance of all models across the two datasets: Hotel Booking Demand (classification) and US Accidents (regression). Each model was evaluated using consistent preprocessing pipelines, stratified splits, and task-appropriate metrics. Runtime and memory usage were recorded to assess scalability and deployment feasibility.

Tables 1 and 2 summarize the key results. For the hotel dataset, all models achieved competitive F1 scores around 0.86, with neural networks and decision trees slightly outperforming support vector machines. The kernel SVM offered strong calibration (ROC-AUC = 0.924) but required significantly more runtime than its linear counterpart. kNN matched the best F1 scores but incurred high memory and prediction latency. Neural networks delivered the highest ROC-AUC values (up to 0.938) with reasonable training time and memory usage, confirming their ability to generalize well in imbalanced classification settings.

For the accident dataset, neural networks again led in performance, with RMSE values below 56 and R^2 exceeding 0.56. Decision trees and kNN provided interpretable baselines with RMSEs around 64–65, while the kernel SVM lagged behind with an RMSE of 77 and R^2 of 0.15. The linear SVM failed to converge, producing extreme error values and negative R^2 , and was excluded from further analysis. A dummy regressor predicting the mean duration yielded an RMSE of 83.28, serving as a reference point for model effectiveness.

These results highlight the trade-offs between interpretability,

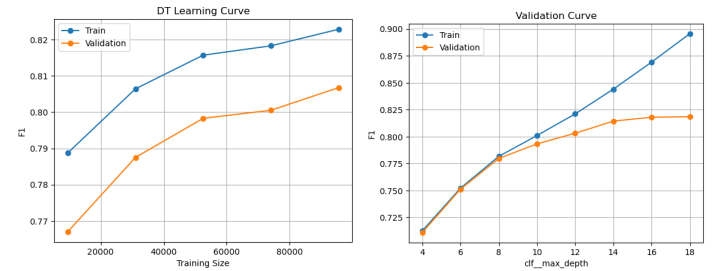
scalability, and predictive power. While tree-based models and kNN offer transparency, neural networks consistently outperform them in accuracy and calibration, especially when tuned with dropout and regularization.

Table 1: Hotel Booking Demand – Classification Results

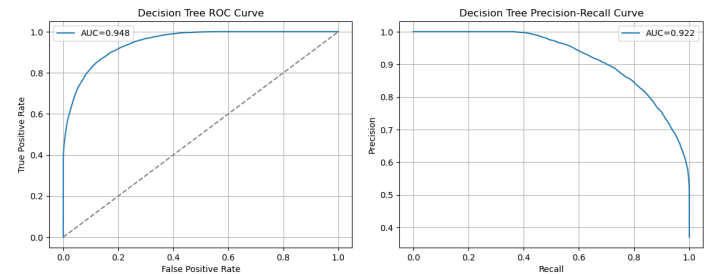
Model	F1	ROC-AUC	Time (min)	RAM (GB)
Decision Tree	0.86	0.947	5.08	0.32
Linear SVM	0.82	–	0.51	0.40
Kernel SVM (RBF)	0.85	0.924	8.64	0.48
kNN	0.86	0.928	3.25	4.25
NN (Shallow)	0.86	0.934	0.11	1.06
NN (Deep)	0.86	0.938	0.08	1.10

Table 2: US Accidents – Regression Results

Model	RMSE	R^2	Time (min)	RAM (GB)
Dummy Baseline	83.28	–	<0.1	<0.1
Decision Tree	64.47	0.397	5.56	3.14
Linear SVM	$> 10^{12}$	-10^{20}	0.03	1.17
Kernel SVM (RBF)	76.97	0.151	9.96	0.75
kNN	65.30	0.391	15.6	4.25
NN (Shallow)	55.98	0.548	8.0	1.06
NN (Deep)	55.13	0.562	11.0	1.10



(a) Learning curve for the Decision Tree classifier. (b) Validation curve with respect to `max_depth`. Training F1 increases steadily, and validation F1 scores improve, indicating that the model benefits from additional data. Training F1 plateaus, suggesting overfitting beyond depth 14.



(c) ROC curve for the Decision Tree classifier. (d) Precision–Recall curve for the Decision Tree classifier. The curve lies well above the diagonal, with an AUC of 0.948, indicating strong discriminative performance. The model achieves a PR-AUC of 0.922, confirming its effectiveness under class imbalance.

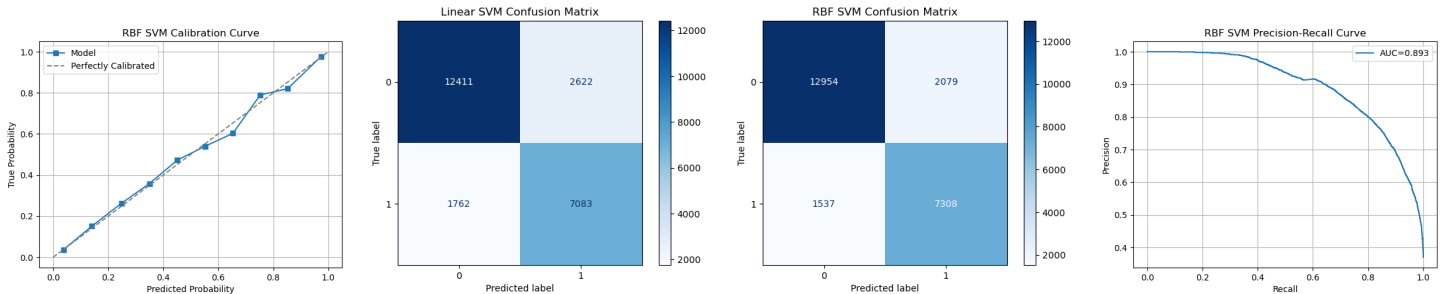
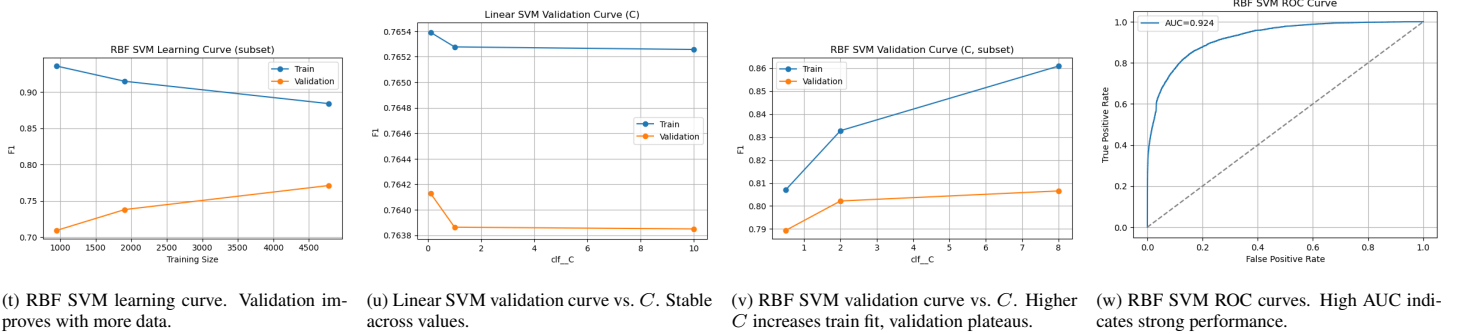
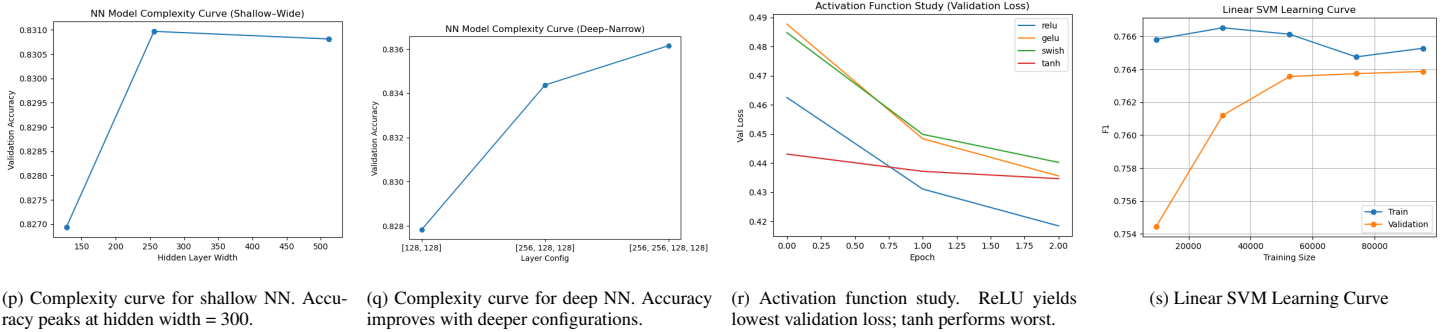
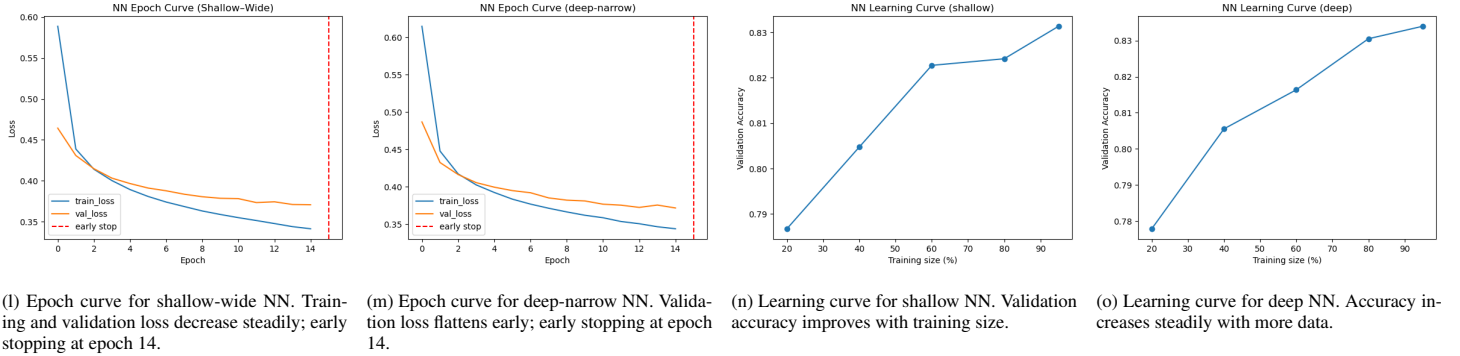
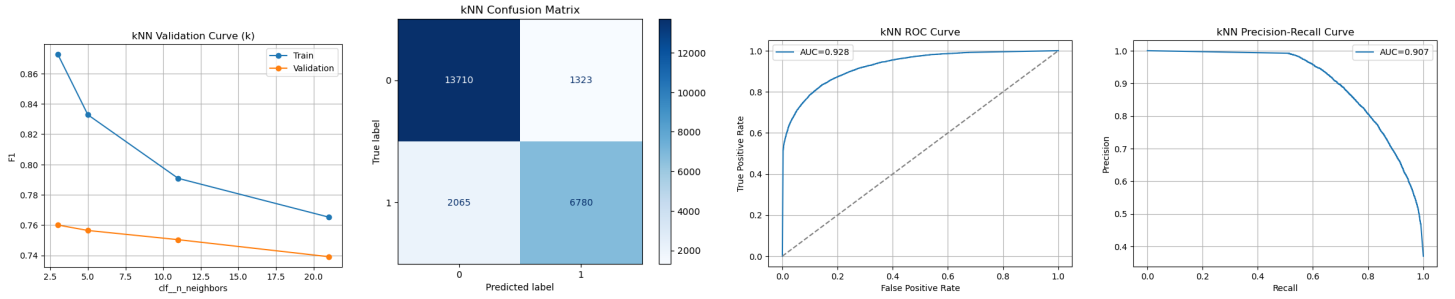


Figure-1: Diagnostic plots for Decision Tree, kNN, Neural Network, and SVM classifiers on the Hotel Booking Demand dataset. The plots summarize learning behavior, validation sensitivity, ROC/PR performance, calibration, and confusion matrices across models.

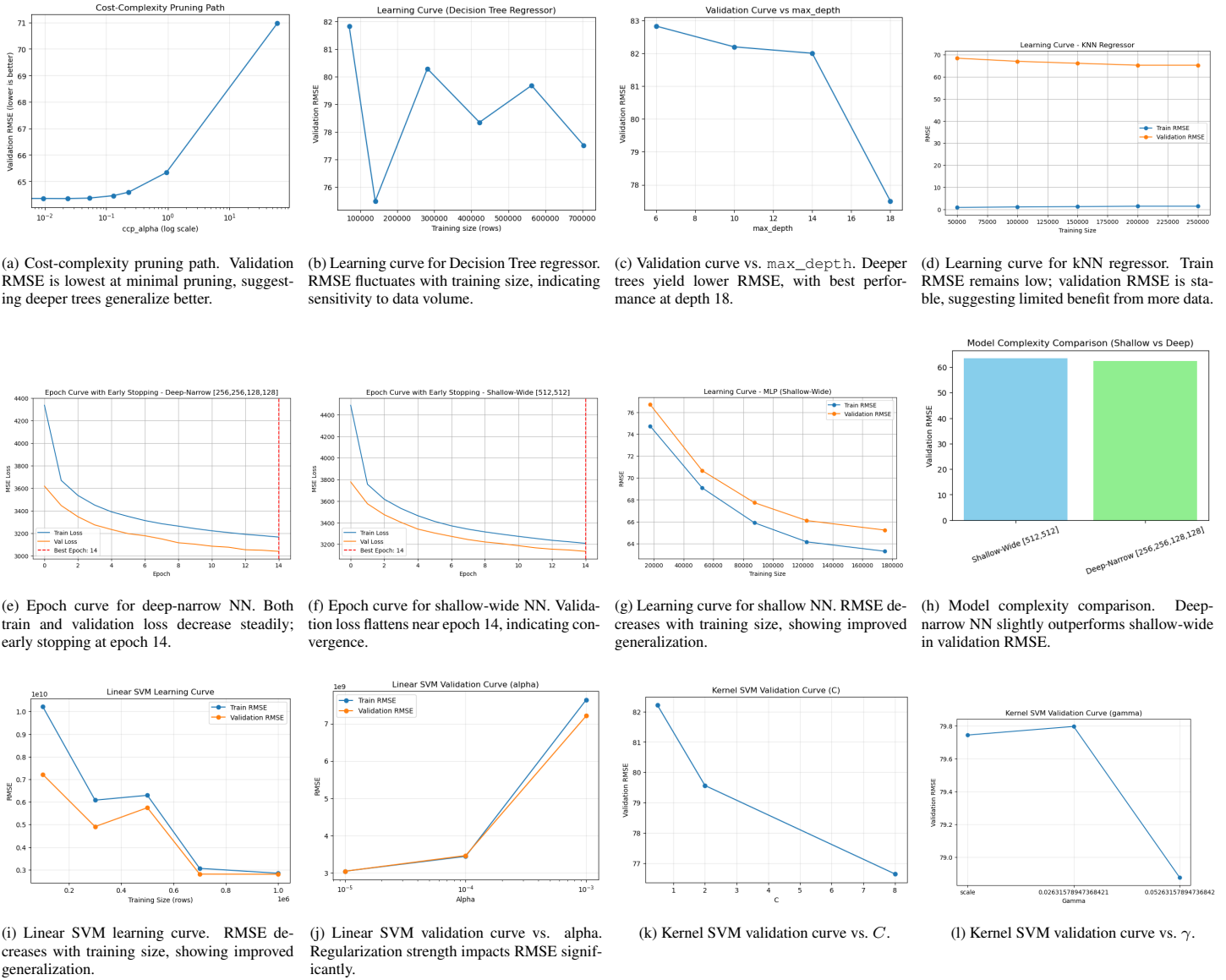


Figure-2: Diagnostic plots for Decision Tree, kNN, Neural Network, and SVM regressors on the US Accidents dataset. The plots summarize pruning, learning behavior, model complexity, and evaluation curves across all models.

5 Discussion

The comparative evaluation across the Hotel Booking Demand and US Accidents datasets highlights several recurring themes regarding model behavior, trade-offs, and practical considerations.

Interpretability vs. Predictive Power: Tree-based models and kNN offered transparent decision boundaries and interpretable diagnostics, making them attractive for stakeholders who prioritize explainability. However, their predictive performance plateaued relative to neural networks, which consistently achieved higher calibration and generalization. This trade-off underscores the tension between interpretability and accuracy in applied machine learning.

Scalability and Resource Constraints. Runtime and memory usage varied substantially across models. kNN incurred high memory costs due to instance-based storage, while kernel SVMs required significant runtime for hyperparameter tuning and probability estimation. In contrast, decision trees and linear SVMs scaled efficiently, though at the expense of predictive accuracy.

Neural networks struck a balance, achieving strong performance with moderate computational demands when regularization and early stopping were applied.

Overfitting and Regularization. Validation and learning curves revealed clear overfitting risks in deeper decision trees and under-regularized neural networks. Techniques such as cost-complexity pruning, dropout, and early stopping proved essential in mitigating these effects. For SVMs, careful tuning of C and γ was critical, as poor choices led to either underfitting (linear SVM) or excessive variance (RBF SVM).

Dataset-Specific Insights. For the Hotel Booking Demand classification task, most models converged to similar F1 scores, but neural networks and decision trees demonstrated superior calibration and robustness under class imbalance. In the US Accidents regression task, neural networks clearly outperformed alternatives, achieving lower RMSE and higher R^2 . The linear SVM failed to converge meaningfully, highlighting the limitations of certain algorithms when applied to large, noisy, real-world regression data.

Failures and Limitations. The most notable failure was the linear SVM on the regression task, which produced extreme error values and negative R^2 . This outcome emphasizes the importance of algorithm–data compatibility. Additionally, while neural networks delivered the strongest results overall, their performance gains were incremental rather than transformative, suggesting diminishing returns without more sophisticated architectures or feature engineering.

Broader Implications. These findings reinforce the importance of aligning model choice with both task requirements and operational constraints. In practice, decision trees and kNN may be preferable when interpretability and low-latency predictions are critical, while neural networks are better suited for maximizing predictive accuracy under moderate resource budgets. Kernel SVMs, though theoretically powerful, proved less practical at scale.

6 Conclusion

This study compared decision trees, k-nearest neighbors, neural networks, and support vector machines across two distinct tasks: hotel booking classification and accident duration regression. Despite differences in data structure and evaluation metrics, several consistent patterns emerged.

Neural networks demonstrated the strongest overall predictive performance, achieving the highest ROC-AUC in classification (up to 0.938, Table 1) and the lowest RMSE in regression (55.13, Table 2). These results support our initial hypothesis that high-capacity nonlinear models would outperform classical baselines, particularly when regularization and early stopping were applied. However, these gains came at the cost of reduced interpretability compared to simpler models.

Decision trees and kNN provided competitive baselines, offering transparency and ease of interpretation. For the hotel dataset, decision trees achieved a ROC-AUC of 0.947 with clear feature importance rankings, while kNN matched neural networks in F1 but incurred high latency. Importantly, the classification task highlighted trade-offs between Type I and Type II errors: false positives (predicting cancellation when none occurred) may reduce revenue through unnecessary overbooking buffers, while false negatives (failing to predict cancellation) risk wasted inventory. For the accident dataset, residual analysis revealed systematic underestimation of long-duration events, suggesting that even the best models struggled with rare but impactful cases.

SVMs showed mixed results: the RBF kernel performed reasonably well in classification (ROC-AUC 0.924) but scaled poorly in regression, while the linear variant diverged entirely. These outcomes reinforce the need to align algorithmic flexibility with dataset scale and structure.

Overall, the findings emphasize the importance of aligning model choice with both task requirements and operational constraints. Neural networks are well-suited for maximizing predictive accuracy under moderate resource budgets, whereas tree-based models and kNN remain valuable when interpretability and efficiency are prioritized. Limitations of this study include runtime-driven subsampling for certain algorithms, the exclusion of ensemble methods such as random forests or gradient boosting, and limited hyperparameter sweeps relative to industrial-scale tuning. Future work could explore hybrid approaches, feature en-

gineering, and ensembles to further balance accuracy, scalability, and transparency.

References

- [1] N. Antonio, A. de Almeida, and L. Nunes, “Hotel booking demand datasets,” *Data in Brief*, vol. 22, pp. 41–49, 2019. doi: 10.1016/j.dib.2018.11.126.
- [2] S. Moosavi, A. Nadimi-Shahraki, and M. H. Nadimi-Shahraki, “US Accidents (2016–2023): A countrywide traffic accident dataset,” in *Proc. IEEE Int. Conf. on Big Data*, 2023. <https://ieeexplore.ieee.org/document/10339651>
- [3] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org/>
- [4] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: <https://www.tensorflow.org/>
- [5] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. doi: 10.1038/s41586-020-2649-2
- [6] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010, pp. 51–56. doi: 10.25080/Majora-92bf1922-00a
- [7] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. doi: 10.1109/MCSE.2007.55
- [8] Microsoft Copilot, “AI assistance for coding and analysis,” Microsoft, Accessed: Sept., 2025. [Online]. Available: <https://copilot.microsoft.com/>

AI Use Statement

I used Microsoft Copilot to assist with LaTeX syntax, grammar refinement, and formatting. Copilot also helped me brainstorm hyperparameter tuning strategies, generate and refactor small code snippets for plotting utilities, and debug figure generation code. I reviewed, verified, and understood all assisted content. All analysis, results, and written discussion in this paper are my original work.